Deccan Airlines — Postman Test Cases

Base URL: `http://localhost:9080/api`

> Run tests in order (top to bottom). Copy the `token` from register/login response and paste it in Authorization header for all protected endpoints.

---

1. AUTHENTICATION

1.1 Register User

```
POST http://localhost:9080/api/auth/register
Content-Type: application/json

Body:
{
  "firstName": "Nedumaaran",
  "lastName": "Rajangam",
  "email": "maara@example.com",
  "password": "Maara@123",
  "phone": "9876543210",
  "dateOfBirth": "1980-05-15",
  "nationality": "Indian",
  "gender": "MALE"
}

Expected: 201 Created
Response has: accessToken, refreshToken, user object
Save: accessToken → use as Bearer token everywhere
Save: refreshToken → use for refresh/logout
```

1.2 Register Duplicate Email (should fail)

```
POST http://localhost:9080/api/auth/register
Content-Type: application/json

Body:
{
  "firstName": "Test",
  "lastName": "User",
```

```
  "email": "maara@example.com",
  "password": "Test@12345",
  "phone": "1111111111"
}

Expected: 400 or 409
Message: "Email already registered"
```

1.3 Register with Weak Password (should fail)

```
POST http://localhost:9080/api/auth/register
Content-Type: application/json

Body:
{
  "firstName": "Test",
  "lastName": "User",
  "email": "weak@example.com",
  "password": "12345678"
}

Expected: 400
Message: Validation failed (missing uppercase, special char)
```

1.4 Login

```
POST http://localhost:9080/api/auth/login
Content-Type: application/json

Body:
{
  "email": "maara@example.com",
  "password": "Maara@123"
}

Expected: 200 OK
Response has: accessToken, refreshToken, user object
```

1.5 Login Wrong Password (should fail)

```
POST http://localhost:9080/api/auth/login
```

```
Content-Type: application/json

Body:
{
  "email": "maara@example.com",
  "password": "WrongPass@1"
}

Expected: 401
Message: "Invalid credentials"
```

1.6 Login Non-existent Email (should fail)

```
POST http://localhost:9080/api/auth/login
Content-Type: application/json

Body:
{
  "email": "nobody@example.com",
  "password": "Maara@123"
}

Expected: 401
```

1.7 Refresh Token

```
POST http://localhost:9080/api/auth/refresh?refreshToken=
<paste_refresh_token_here>

Expected: 200 OK
Response has: new accessToken, new refreshToken
```

1.8 Refresh with Invalid Token (should fail)

```
POST http://localhost:9080/api/auth/refresh?refreshToken=invalid-token-12345

Expected: 400
Message: "Invalid refresh token"
```

---

2. SECURITY QUESTION & FORGOT PASSWORD

2.1 Get All Security Questions

```
GET http://localhost:9080/api/auth/security-questions

Expected: 200 OK
Response: Array of 8 questions with questionId and questionText
No auth required
```

2.2 Set Security Question (requires token)

```
POST http://localhost:9080/api/auth/security-question
Content-Type: application/json
Authorization: Bearer <paste_token_here>

Body:
{
  "questionId": 2,
  "answer": "Chennai"
}

Expected: 200 OK
Message: "Question set"
```

2.3 Set Security Question Without Token (should fail)

```
POST http://localhost:9080/api/auth/security-question
Content-Type: application/json

Body:
{
  "questionId": 2,
  "answer": "Chennai"
}

Expected: 401 or 403
```

2.4 Forgot Password — Get Question

```
GET http://localhost:9080/api/auth/forgot-password?email=maara@example.com
```

```
Expected: 200 OK
Response: { "questionId": 2, "questionText": "What city were you born in?" }
No auth required
```

2.5 Forgot Password — Non-existent Email (should fail)

```
GET http://localhost:9080/api/auth/forgot-password?email=nobody@example.com

Expected: 404
Message: "No account found"
```

2.6 Verify Security Answer — Correct

```
POST http://localhost:9080/api/auth/verify-security-answer
Content-Type: application/json

Body:
{
  "email": "maara@example.com",
  "questionId": 2,
  "answer": "Chennai"
}

Expected: 200 OK
Message: "Answer verified"
```

2.7 Verify Security Answer — Wrong (should fail)

```
POST http://localhost:9080/api/auth/verify-security-answer
Content-Type: application/json

Body:
{
  "email": "maara@example.com",
  "questionId": 2,
  "answer": "Mumbai"
}

Expected: 401
Message: "Incorrect answer. 4 attempts remaining."
```

2.8 Reset Password

```
POST http://localhost:9080/api/auth/reset-password
Content-Type: application/json

Body:
{
  "email": "maara@example.com",
  "questionId": 2,
  "answer": "Chennai",
  "newPassword": "NewMaara@456"
}

Expected: 200 OK
Message: "Password reset successful"
```

2.9 Login with New Password

```
POST http://localhost:9080/api/auth/login
Content-Type: application/json

Body:
{
  "email": "maara@example.com",
  "password": "NewMaara@456"
}

Expected: 200 OK
Save: new accessToken for remaining tests
```

2.10 Login with Old Password (should fail)

```
POST http://localhost:9080/api/auth/login
Content-Type: application/json

Body:
{
  "email": "maara@example.com",
  "password": "Maara@123"
}
```

```
Expected: 401
```

---

3. USER PROFILE

3.1 Get My Profile

```
GET http://localhost:9080/api/users/me
Authorization: Bearer <token>

Expected: 200 OK
Response: userId, firstName, lastName, email, phone, role, etc.
```

3.2 Get Profile Without Token (should fail)

```
GET http://localhost:9080/api/users/me

Expected: 401 or 403
```

3.3 Update Profile

```
PUT http://localhost:9080/api/users/me
Content-Type: application/json
Authorization: Bearer <token>

Body:
{
  "phone": "9999888877",
  "passportNumber": "B98765432",
  "nationality": "Indian"
}

Expected: 200 OK
Response: Updated user with new phone and passport
```

3.4 Change Password

```
PUT http://localhost:9080/api/users/me/password
Content-Type: application/json
```

```
Authorization: Bearer <token>

Body:
{
  "currentPassword": "NewMaara@456",
  "newPassword": "Maara@789"
}

Expected: 200 OK
Message: "Password changed"
```

3.5 Change Password — Wrong Current (should fail)

```
PUT http://localhost:9080/api/users/me/password
Content-Type: application/json
Authorization: Bearer <token>

Body:
{
  "currentPassword": "WrongPassword@1",
  "newPassword": "Something@123"
}

Expected: 400
Message: "Current password is incorrect"
```

---

4. FLIGHTS (No auth required)

4.1 Search Flights — MAA to BOM

```
GET http://localhost:9080/api/flights/search?from=MAA&to=BOM&date=2026-04-
22&passengers=1

NOTE: Change date to tomorrow's date

Expected: 200 OK
Response: Array with flight DA 203
```

4.2 Search Flights — No Results

```
GET http://localhost:9080/api/flights/search?from=MAA&to=XYZ&date=2026-04-
22&passengers=1

Expected: 200 OK
Response: Empty array []
```

4.3 Search Flights — Too Many Passengers

```
GET http://localhost:9080/api/flights/search?from=MAA&to=BOM&date=2026-04-
22&passengers=999

Expected: 200 OK
Response: Empty array [] (no flight has 999 available seats)
```

4.4 Get Flight by ID

```
GET http://localhost:9080/api/flights/101

Expected: 200 OK
Response: Flight DA 203, MAA → BOM
```

4.5 Get Flight by Invalid ID (should fail)

```
GET http://localhost:9080/api/flights/9999

Expected: 404
Message: "Flight not found: 9999"
```

4.6 Get Tatkal Flights

```
GET http://localhost:9080/api/flights/tatkal

Expected: 200 OK
Response: Array of flights departing within 24 hours with tatkal seats > 0
```

4.7 Get Seat Map

```
GET http://localhost:9080/api/flights/101/seats

Expected: 200 OK
Response: Array of 21 seats (15 regular + 6 tatkal)
Each seat has: seatId, seatNumber, cabinClass, seatType, isAvailable, isTatkal
```

---

5. REGULAR BOOKING (requires auth)

5.1 Create Booking — 1 Passenger

```
POST http://localhost:9080/api/bookings
Content-Type: application/json
Authorization: Bearer <token>

Body:
{
  "flightId": 101,
  "passengers": [
    {
      "firstName": "Maara",
      "lastName": "Rajangam",
      "age": 45,
      "gender": "MALE",
      "passportNumber": "A12345678",
      "seatId": 5001
    }
  ]
}

Expected: 201 Created
Response: bookingId, pnr (starts with R), status=PENDING, bookingType=REGULAR
Save: bookingId and pnr
```

5.2 Create Booking — 2 Passengers

```
POST http://localhost:9080/api/bookings
Content-Type: application/json
Authorization: Bearer <token>
```

```
Body:
{
  "flightId": 101,
  "passengers": [
    {
      "firstName": "Ravi",
      "lastName": "Kumar",
      "age": 30,
      "gender": "MALE",
      "seatId": 5004
    },
    {
      "firstName": "Priya",
      "lastName": "Kumar",
      "age": 28,
      "gender": "FEMALE",
      "seatId": 5005
    }
  ]
}

Expected: 201 Created
totalPassengers: 2
totalFare: 4599.00 × 2 = 9198.00
```

5.3 Book Already Taken Seat (should fail)

```
POST http://localhost:9080/api/bookings
Content-Type: application/json
Authorization: Bearer <token>

Body:
{
  "flightId": 101,
  "passengers": [
    {
      "firstName": "Test",
      "lastName": "User",
      "age": 25,
      "gender": "MALE",
      "seatId": 5001
    }
```

```
  ]
}

Expected: 400 or 409
Message: "Seat not available: 5001"
```

5.4 Book Business Class Seat

```
POST http://localhost:9080/api/bookings
Content-Type: application/json
Authorization: Bearer <token>

Body:
{
  "flightId": 101,
  "passengers": [
    {
      "firstName": "Vikram",
      "lastName": "Mehta",
      "age": 50,
      "gender": "MALE",
      "seatId": 5010
    }
  ]
}

Expected: 201 Created
totalFare: 4599.00 × 1.80 = 8278.20 (business class modifier)
```

5.5 Confirm Payment

```
POST http://localhost:9080/api/bookings/confirm-payment
Content-Type: application/json
Authorization: Bearer <token>

Body:
{
  "bookingId": <paste_bookingId_from_5.1>,
  "paymentMethod": "CREDIT_CARD",
  "transactionId": "TXN-REG-001"
}
```

```
Expected: 200 OK
status: "CONFIRMED"
```

5.6 Confirm Payment Again (should fail)

```
POST http://localhost:9080/api/bookings/confirm-payment
Content-Type: application/json
Authorization: Bearer <token>

Body:
{
  "bookingId": <same_bookingId>,
  "paymentMethod": "UPI",
  "transactionId": "TXN-REG-002"
}

Expected: 400
Message: "Booking not PENDING"
```

5.7 Get Booking by PNR

```
GET http://localhost:9080/api/bookings/pnr/<paste_pnr_here>
Authorization: Bearer <token>

Expected: 200 OK
Response: Full booking with passengers array
```

5.8 Get Booking by ID

```
GET http://localhost:9080/api/bookings/<paste_bookingId>
Authorization: Bearer <token>

Expected: 200 OK
Response: Full booking with passengers, seatNumber, cabinClass
```

5.9 Get My Bookings

```
GET http://localhost:9080/api/bookings/my
Authorization: Bearer <token>
```

```
Expected: 200 OK
Response: Array of all bookings for this user
```

5.10 Cancel Booking

```
POST http://localhost:9080/api/bookings/<paste_bookingId>/cancel
Authorization: Bearer <token>

Expected: 200 OK
Response: { "bookingId": ..., "status": "CANCELLED" }
```

5.11 Cancel Already Cancelled (should fail)

```
POST http://localhost:9080/api/bookings/<same_bookingId>/cancel
Authorization: Bearer <token>

Expected: 400
Message: "Already cancelled"
```

---

6. TATKAL BOOKING (requires auth)

6.1 Create Tatkal Booking

```
POST http://localhost:9080/api/tatkal/book
Content-Type: application/json
Authorization: Bearer <token>

Body:
{
  "flightId": 101,
  "passengers": [
    {
      "firstName": "Anjali",
      "lastName": "Sharma",
      "age": 35,
      "gender": "FEMALE"
    }
  ]
}
```

```
NOTE: Flight 101 must depart within 24 hours for this to work

Expected: 201 Created
pnr: starts with "T"
bookingType: "TATKAL"
refundPolicy: "NON-REFUNDABLE"
totalFare: 4599.00 × 1.30 = 5978.70 (tatkal multiplier)
Seat auto-assigned (no seatId in request)
Save: bookingId
```

6.2 Tatkal with 2 Passengers

```
POST http://localhost:9080/api/tatkal/book
Content-Type: application/json
Authorization: Bearer <token>

Body:
{
  "flightId": 101,
  "passengers": [
    {
      "firstName": "Kiran",
      "lastName": "Patel",
      "age": 40,
      "gender": "MALE"
    },
    {
      "firstName": "Meena",
      "lastName": "Patel",
      "age": 38,
      "gender": "FEMALE"
    }
  ]
}

Expected: 201 Created
totalPassengers: 2
```

6.3 Tatkal — Too Many Passengers (should fail)

```
POST http://localhost:9080/api/tatkal/book
Content-Type: application/json
```

```
Authorization: Bearer <token>

Body:
{
  "flightId": 101,
  "passengers": [
    {"firstName":"A","lastName":"B","age":30,"gender":"MALE"},
    {"firstName":"C","lastName":"D","age":30,"gender":"MALE"},
    {"firstName":"E","lastName":"F","age":30,"gender":"MALE"},
    {"firstName":"G","lastName":"H","age":30,"gender":"MALE"},
    {"firstName":"I","lastName":"J","age":30,"gender":"MALE"}
  ]
}

Expected: 400
Message: "Max 4 passengers per Tatkal booking"
```

6.4 Tatkal — Flight Outside Window (should fail)

```
POST http://localhost:9080/api/tatkal/book
Content-Type: application/json
Authorization: Bearer <token>

Body:
{
  "flightId": 104,
  "passengers": [
    {
      "firstName": "Test",
      "lastName": "User",
      "age": 30,
      "gender": "MALE"
    }
  ]
}

NOTE: Flight 104 departs day-after-tomorrow (outside 24h window)

Expected: 403
Message: "Tatkal not open yet"
```

6.5 Confirm Tatkal Payment

```
POST http://localhost:9080/api/tatkal/confirm-payment
Content-Type: application/json
Authorization: Bearer <token>

Body:
{
  "bookingId": <paste_tatkal_bookingId_from_6.1>,
  "paymentMethod": "UPI",
  "transactionId": "TXN-TAT-001"
}

Expected: 200 OK
status: "CONFIRMED"
```

6.6 Cancel Tatkal Booking (non-refundable)

```
POST http://localhost:9080/api/bookings/<tatkal_bookingId>/cancel
Authorization: Bearer <token>

Expected: 200 OK
Response: { "bookingId": ..., "status": "CANCELLED" }
NOTE: refundAmount will be 0 (Tatkal is non-refundable)
```

---

7. MEALS (requires auth)

7.1 Get Full Menu

```
GET http://localhost:9080/api/meals/menu
Authorization: Bearer <token>

Expected: 200 OK
Response: Array of 10 meal items
```

7.2 Get VEG Menu Only

```
GET http://localhost:9080/api/meals/menu?mealType=VEG
Authorization: Bearer <token>
```

```
Expected: 200 OK
Response: Array with only VEG items (Paneer Tikka Masala, South Indian Meals)
```

7.3 Get VEGAN Menu

```
GET http://localhost:9080/api/meals/menu?mealType=VEGAN
Authorization: Bearer <token>

Expected: 200 OK
Response: Array with Vegan Buddha Bowl
```

7.4 Get Invalid Meal Type (should fail)

```
GET http://localhost:9080/api/meals/menu?mealType=INVALID
Authorization: Bearer <token>

Expected: 400
```

7.5 Create Meal Order

```
POST http://localhost:9080/api/meals/orders
Content-Type: application/json
Authorization: Bearer <token>

Body:
{
  "bookingId": <any_valid_bookingId>,
  "passengerId": 1,
  "passengerName": "Maara Rajangam",
  "mealItemId": 1,
  "quantity": 1,
  "specialInstructions": "Extra spicy please"
}

Expected: 201 Created
Response: orderId, mealItem details, totalPrice=250.00, status=CONFIRMED
Save: orderId
```

7.6 Create Meal Order — Quantity 3

```
POST http://localhost:9080/api/meals/orders
Content-Type: application/json
Authorization: Bearer <token>

Body:
{
  "bookingId": <any_valid_bookingId>,
  "passengerId": 1,
  "passengerName": "Maara Rajangam",
  "mealItemId": 2,
  "quantity": 3,
  "specialInstructions": null
}

Expected: 201 Created
totalPrice: 350.00 × 3 = 1050.00
```

7.7 Create Meal Order — Invalid Meal ID (should fail)

```
POST http://localhost:9080/api/meals/orders
Content-Type: application/json
Authorization: Bearer <token>

Body:
{
  "bookingId": 1,
  "passengerId": 1,
  "passengerName": "Test User",
  "mealItemId": 9999,
  "quantity": 1
}

Expected: 400
Message: "Meal not found"
```

7.8 Update Meal Order

```
PUT http://localhost:9080/api/meals/orders/<paste_orderId>
Content-Type: application/json
Authorization: Bearer <token>
```

```
Body:
{
  "mealItemId": 3,
  "quantity": 2,
  "specialInstructions": "No sesame"
}

Expected: 200 OK
status: "MODIFIED"
totalPrice recalculated: 280.00 × 2 = 560.00
```

7.9 Update Only Quantity

```
PUT http://localhost:9080/api/meals/orders/<paste_orderId>
Content-Type: application/json
Authorization: Bearer <token>

Body:
{
  "quantity": 1
}

Expected: 200 OK
totalPrice recalculated with current meal item
```

7.10 Get Meal Orders by Booking

```
GET http://localhost:9080/api/meals/orders/booking/<paste_bookingId>
Authorization: Bearer <token>

Expected: 200 OK
Response: Array of all non-cancelled meal orders for that booking
```

7.11 Get Meal Orders by Passenger

```
GET http://localhost:9080/api/meals/orders/passenger/1
Authorization: Bearer <token>

Expected: 200 OK
Response: Array of meal orders for passenger ID 1
```

7.12 Cancel Meal Order

```
DELETE http://localhost:9080/api/meals/orders/<paste_orderId>
Authorization: Bearer <token>

Expected: 200 OK
Message: "Cancelled"
```

7.13 Cancel Already Cancelled (should fail)

```
DELETE http://localhost:9080/api/meals/orders/<same_orderId>
Authorization: Bearer <token>

Expected: 400
Message: "Order is already cancelled" or similar
```

---

8. LOGOUT

8.1 Logout

```
POST http://localhost:9080/api/auth/logout?refreshToken=<paste_refresh_token>

Expected: 200 OK
Message: "Logged out"
```

8.2 Use Expired Refresh Token (should fail)

```
POST http://localhost:9080/api/auth/refresh?refreshToken=<same_token_after_logout>

Expected: 400
Message: "Invalid refresh token"
```

QUICK REFERENCE — Test Order

Step	Test	What to save
1	Register (1.1)	token, refreshToken
2	Set security question (2.2)	—
3	Search flights (4.1)	—
4	View seats (4.7)	pick seatId
5	Create booking (5.1)	bookingId, pnr
6	Confirm payment (5.5)	—
7	Create tatkal (6.1)	tatkalBookingId
8	Confirm tatkal payment (6.5)	—
9	Order meal (7.5)	mealOrderId
10	View my bookings (5.9)	—
11	Cancel booking (5.10)	—
12	Logout (8.1)	—

Total: 48 test cases covering all endpoints, happy paths, and error scenarios.